Automating Privilege Escalation with Deep Reinforcement Learning

Kalle Kujanpää
kalle.kujanpaa@aalto.fi
Aalto University
Espoo, Finland

Willie Victor
willie.victor@f-secure.com
F-Secure
Johannesburg, South Africa

Alexander Ilin
alexander.ilin@aalto.fi
Aalto University
Espoo, Finland

ABSTRACT

AI-based defensive solutions are necessary to defend networks and information assets against intelligent automated attacks. Gathering enough realistic data for training machine learning-based defenses is a significant practical challenge. An intelligent red teaming agent capable of performing realistic attacks can alleviate this problem. However, there is little scientific evidence demonstrating the feasibility of fully automated attacks using machine learning. In this work, we exemplify the potential threat of malicious actors using deep reinforcement learning to train automated agents. We present an agent that uses a state-of-the-art reinforcement learning algorithm to perform local privilege escalation. Our results show that the autonomous agent can escalate privileges in a Windows 7 environment using a wide variety of different techniques depending on the environment configuration it encounters. Hence, our agent is usable for generating realistic attack sensor data for training and evaluating intrusion detection systems.

CCS CONCEPTS

• **Computing methodologies** → **Reinforcement learning**; *Neural networks*; *Partially-observable Markov decision processes*; • **Security and privacy** → *Network security*; *Intrusion detection systems*; • **Applied computing** → System forensics.

KEYWORDS

A2C, actor-critic, attack automation, autonomous cyber defense, autonomous malware, deep reinforcement learning, red teaming, neural networks, POMDP, post-exploitation, privilege escalation

ACM Reference Format:
Kalle Kujanpää, Willie Victor, and Alexander Ilin. 2021. Automating Privilege Escalation with Deep Reinforcement Learning. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security (AISec '21), November 15, 2021, Virtual Event, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3474369.3486877

1 INTRODUCTION

Defending networks and information assets from attack in a constantly evolving threat landscape remains a substantial challenge in our modern and connected world. As better detection and response methods are developed, attackers invariably adapt their tools and techniques to remain competitive. One adaptation is the use of advanced automation to perform attack sequences so quickly they cannot be responded to by defenders. An example of this is the NotPetya malware from 2017. The malware spread very rapidly by credential dumping and lateral movement techniques usually expected of human-on-keyboard attacks (see e.g. [20]).

Partial or even full automation of attacks is nothing new. Many exploits and techniques consist of discrete steps, and they can be easily scripted. Frameworks such as Metasploit, OpenVAS, Cobalt Strike, and PowerShell Empire support and automate red teaming activities. However, this type of automation relies on several assumptions about the target environment and often requires a human to configure it for a specific scenario. Moreover, the highly predictable sequences of observable events generated by automated attacks give a further advantage to defenders. Hence, these kinds of attacks can often be detected and responded to efficiently. In contrast, a human expert might be capable of determining the optimal approach for a given target almost immediately and avoid detection, thanks to the experience that they have gathered.

Can the human approach be emulated by intelligent machine learning agents that learn from experience, instead of the user having to enumerate all possibilities and creating logic trees to account for all the alternatives? There would be many potential use cases for an intelligent machine learning red teaming agent. To train comprehensive defensive systems using machine learning, enormous amounts of data of realistic attacks might be necessary. An intelligent agent enables the generation of large amounts of attack data, on-demand, that could be used to train or refine detection models. Moreover, in order to understand how to defend against attacks and perform risk assessment, the potential behavior and threats posed by these machine learning-based agents must be understood by the cyber security community.

Despite a fear of malicious actors using reinforcement learning for offensive purposes and the significant advances in deep reinforcement learning during the past few years, few studies on red teaming with reinforcement learning have been published. The most likely reason for this is that the problem of learning to perform an attack is extremely hard:

- a complete attack typically consists of a long sequence of interdependent steps;
- the action space is practically infinite if the actions are the commands that the agent can execute;
- even formalizing red teaming activities as machine learning problems can be extremely challenging.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
AISec '21, November 15, 2021, Virtual Event, Republic of Korea
© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8657-9/21/11…$15.00
https://doi.org/10.1145/3474369.3486877

Therefore, existing research has focused on automating smaller sub-tasks such as initial access [37] or lateral movement during post-exploitation [28].

DeepExploit [37] is a deep reinforcement learning agent that is trained to automate gaining initial access using known vulnerabilities and exploits. It is built on the Metasploit framework [35]. After a successful penetration, it tries to recursively gain access to other hosts in the local network of the given input IP address. DeepExploit is primarily a framework for penetration testing, and its support for post-exploitation activities is very limited: the agent treats lateral movement as a second initial access task.

The deep RL agent of Maeda and Mimura [28] is a step forward in emulating adversarial behavior in real environments: it is trained to perform lateral movement in Windows domains. The authors train the agent using the modules of PowerShell Empire as the action space. The state of the proposed agent consists of ten entries, such as the number of discovered computers in the network, the number of compromised computers, and whether the agent has local administrative privileges. The authors demonstrate that the reinforcement learning agent can learn to perform lateral movement and obtain domain controller privileges.

In this work, we present one potential use case for an intelligent machine learning red teaming agent: we use deep RL to automate the task of local privilege escalation. Privilege escalation is the typical first step performed by an attacker after gaining initial access, and it is often followed by lateral movement to other hosts in the penetrated network. We consider privilege escalation in Windows 7 environments, which may have an arbitrary number of system components (such as services, DLLs, and tasks). We propose a formalization of the privilege escalation task as a reinforcement learning problem and present a novel architecture of an actor-critic RL agent. We experimentally show that the proposed agent can learn to perform the privilege escalation task.

Although we focus on one sub-task performed by a malicious actor, the learning problem that we consider is significantly harder compared to previous works [28, 37]:

- Privilege escalation needs a sequence of actions, the selection of which depends on the changing system state. The scenario of [37], for example, has one-step solutions without any changes in the system state.
- Privilege escalation can be accomplished by multiple different strategies.
- The attacked system can have a varying number of system components (services, DLLs, tasks), and the agent should generalize to any number of those.
- Our training setup is more realistic and diverse compared to [28]. Instead of attacking a system whose variability is implemented by noise in system parameters, we use different system configurations in each training episode.
- The state of the system in our experiments is described by thousands of variables, which is much larger than the states of the agents in [28, 37].
- The actions that we design for our learning environment are more atomic compared to [28, 37]. Most of our actions can be implemented with standard OS commands instead of using modules of existing exploitation frameworks.

Thus, our study takes a step towards solving more complex red teaming tasks with artificial intelligence.

2 RELATED WORK

Applying reinforcement learning in cyber security has been a subject of much recent research [32]. Examples of application areas include, among others, anti-jamming communication systems [21], spoofing detection in wireless networks [41], phishing detection [8], autonomous cyber defense in software-defined networking [22], mobile cloud offloading for malware detection [39], botnet detection [1], security in mobile edge caching [42], and security in autonomous vehicle systems [16]. In addition, reinforcement learning has been applied to research of physical security, such as grid security [33], and to green security games [40].

Previously, multi-agent reinforcement learning has been applied to cyber security simulations with competing adversarial and defensive agents [5, 12, 23]. It has been shown that both the attacking and the defending reinforcement learning agents can learn to improve their performance. The success of multi-agent reinforcement learning might have wider implications for information security research even though these simulation-based studies are not directly applicable to real environments.

There have also been attempts to apply reinforcement learning to penetration testing [6, 10, 18, 19, 37, 43]. The results of these efforts suggest that reinforcement learning can support the human in charge of the penetration testing process [6, 18]. Reinforcement learning has also been applied to planning the steps following the initial penetration by learning a policy in a simulated version of the environment [10]. Penetration testing [43] and web hacking [13] have also been converted to simulated capture-the-flag challenges that can be solved with reinforcement learning. Finally, reinforcement learning has been applied to attacking static Portable Executable (PE) [2] and even supervised learning-based anti-malware engines [15]. The trained agents are capable of modifying the malware to evade detection.

Reinforcement learning has also been applied to blue teaming. Microsoft has developed a research toolkit called CyberBattleSim, which enables modeling the behavior of autonomous agents in a high-level abstraction of a computer network [38]. Reinforcement learning agents that operate in the abstracted network can be trained using the framework. The objective of the platform is to create an understanding of how malicious reinforcement learning agents could behave in a network and how reinforcement learning can be used for threat detection. Deep reinforcement learning can also be applied to improving feature selection for malware detection [14].

Non-learning-based approaches to automating adversary emulation have been developed as well. Caldera is a framework capable of automatically planning adversarial actions against Windows enterprise networks. Caldera uses an inbuilt model of the structure of enterprise domains and knowledge about the objectives and potential actions of an attacker. Then, an intelligent heuristics-based planner decides which adversarial actions to perform [3]. Moreover, several different non-RL-based AI approaches to penetration testing and vulnerability analysis have been proposed [29].

Supervised and unsupervised learning with and without neural networks have been applied for blue teaming purposes. For instance, malicious PowerShell commands can be detected with novel deep learning methods [24]. An ensemble detector combining an NLP-based classifier with a CNN-based classifier was the best at detecting malicious commands. The detection performance was high enough to be useful in practice. The detector was evaluated using a large dataset consisting of legitimate commands executed by standard users, malicious commands executed by malware, and malicious commands designed by security experts. The suitability of machine learning for intrusion detection, malicious code detection, malware analysis, and spam detection has been discussed [4, 11, 26]. These methods often rely on extensive feature engineering [7, 26]. In defensive tasks, good performance can often be achieved without deep neural networks, with solutions like logistic regression, support vector machines, and random forests [30]. Machine learning-based systems are vulnerable to adversarial attacks [9], and as different ML-based techniques have dissimilar weaknesses, a combination of machine learning techniques is often necessary [4, 7].

3 REINFORCEMENT LEARNING

Reinforcement learning is one of the three main paradigms of machine learning alongside supervised and unsupervised learning [36]. In reinforcement learning, an agent interacts with an environment over discrete time steps to maximize its long-run reward. At a given time step t, the environment has state s_t^e and the agent is given an observation $o_t = f(s_t^e)$ and a reward signal r_t. If the environment is fully observable, the observation is equal to the environment state, $o_t = s_t^e$. In a more general scenario, the agent receives only a partial observation o_t which does not represent the full environment state. In this case, the agent has its own state s_t which might differ from the environment state s_t^e. The agent selects an action a_t from the set of possible actions A and acts in the environment. The environment transitions to a new state s_{t+1}^e and the agent receives a new observation o_{t+1} and a new reward r_{t+1}. The goal of the agent is to maximize the sum of the collected rewards $R = \sum_{t=0}^{\infty} \gamma^t r_t$ where $\gamma \in (0, 1]$ is a discount factor used to discount future rewards [31].

In this paper, we use a model-free approach to reinforcement learning in which the agent does not build an explicit model of the environment. The agent selects an action a_t according to a policy function $\pi(a_t|s_t)$ which depends on the agent state s_t. We use an algorithm called the advantage actor-critic (A2C) [31] in which the policy is parameterized as $\pi(a|s;\theta)$. The parameters θ of the policy are updated in the direction of

$$\nabla_\theta \log \pi(a_t|s_t;\theta) A(a_t, s_t) \tag{1}$$

where $A(a_t, s_t)$ is an advantage function which estimates the (relative) benefit of taking action a_t in state s_t in terms of the expected total reward. In A2C, the advantage function is computed as

$$A(a_t, s_t) = \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}) - V(s_t)$$

where $V(s)$ is the state-value function which estimates the expected total reward when the agent starts at state s_t and follows policy $\pi(a_t|s_t;\theta)$:

$$V(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s\right].$$

We update the parameters ω of the value function $V(s_t;\omega)$ using Monte Carlo estimates of the total discounted rewards as the targets

$$\text{loss}\left(V(s_t;\omega), \sum_{k=0}^{\infty} \gamma^k r_{t+k}\right) \to \min_{\omega}$$

using the Huber loss [25]. In practice, most of the parameters θ and ω are shared (see Section 5.2 and Figure 1).

4 PRIVILEGE ESCALATION AS A REINFORCEMENT LEARNING TASK

4.1 Problem Definition

In this work, we focus on automating one particular step often performed by red teaming actors: local privilege escalation in Windows. For our reinforcement learning agent, there will be three possible paths to success:

- Add the current user as a local administrator
- Obtain administrative credentials
- Overwrite a program that is executed with elevated privileges when a user or an administrator logs on

The first alternative is hardly how a true red teaming actor would approach the problem as changes in the local administrators of a workstation are easily detectable by any advanced detection and response system. However, if the agent is successful at doing that, it demonstrates that the agent can, with some exceptions, execute arbitrary code with elevated privileges on the victim host. The second alternative is a more realistic alternative for performing local privilege escalation. The third method is arguably inferior to the other two methods as it requires the attacker to wait for the system to be rebooted or some other event to occur that triggers the scheduled task or the AutoRun.

4.2 Learning Environment

The learning environment is a simulated Windows 7 environment with a random non-zero number of services, tasks, and AutoRuns. In each training episode, we introduce one vulnerability in the simulated system by selecting randomly from the following 12 alternatives:

(1) hijackable DLL
  - missing DLL
  - writable DLL
(2) re-configurable service
(3) unquoted service path
(4) modifiable ImagePath in the service registry
(5) writable executable pointed to by a service
(6) missing service binary and a writable service folder
(7) writable binary pointed to by an AutoRun
(8) alwaysInstallElevated bits set to one
(9) credentials of a user with elevated access in the WinLogon registry
(10) credentials of a user with elevated access in an Unattend file

(11) writable binary pointed to by a scheduled task running with elevated privileges
(12) writable Startup folder

To increase the variability of the environment states, we also randomly add services, tasks, and AutoRuns that might initially seem vulnerable to the agent. For instance, a service with one of the service-specific vulnerabilities above but without elevated privileges or a service with a writable parent folder but without an unquoted path can be added. Moreover, standard user credentials might be added to the registry, or a folder on the Windows path might be made writable, among others.

To train an autonomous reinforcement learning agent to perform local privilege escalation, we need to formalize the learning problem, that is, we need to define the reward function r, the action space A, and the space of the agent states.

Defining the reward function is perhaps the easiest task. We selected the simplest possible reward structure without any reward shaping. The agent is given a reward $r = 1$ for the final action of the episode if the privilege escalation has been performed successfully. Otherwise, a zero reward is given. Based on our experiments, this simple sparse reward signal is sufficient for teaching the agent to perform privilege escalation with as few actions as possible because the reward is progressively discounted as more steps are taken by the agent. We also experimented by giving the agent only half a reward ($+\frac{1}{2}$) for performing privilege escalation by the third, arguably inferior method, but the agent had trouble learning the desired behavior.

The state s_t^e of the environment and its dynamics are determined by the Windows 7 environment (or its simulator) that the agent interacts with. The environment is only partially observable: the observations o_t are the outputs of the commands that the agent executes. Working with such a rich observation space is difficult, and therefore, we have designed a custom procedure that converts the observations into the agent state s_t. It is the agent state that is used as the input of the policy $\pi(a_t|s_t)$ and value $V(s_t)$ functions. We also manually designed a set A of high-level actions that the agent needs to choose from. We describe the agent state and the action space in the following sections.

Table 1: General information about the system stored in the agent state

Trinary variables:
(1) Are there credentials in files?
(2) Do the credentials in the files belong to users with elevated privileges?
(3) Are there credentials in the registry?
(4) Do the credentials in the registry belong to users with elevated privileges?
(5) Is there a writable folder on the Windows path?
(6) Are the AlwaysInstallElevated bits set?
(7) Can the AutoRuns be enumerated using an external PowerShell module?
Binary variables:
Has a malicious executable been created in Kali Linux?
Has a malicious service executable been created in Kali Linux?
Has a malicious DLL been created in Kali Linux?
Has a malicious MSI file been created in Kali Linux
Has a malicious executable been downloaded?
Has a malicious service executable been downloaded?
Has a malicious DLL been downloaded?
Has a malicious MSI file been downloaded?
Does the agent know the list of local users?
Are there users whose privileges need to be checked?
Does the agent know the services running on the OS?
Does the agent know the scheduled tasks running on the OS?
Does the agent know the AutoRuns of the OS?
Has the agent performed a static analysis of the service binaries to detect DLLs?
Have the DLLs loaded by the service binaries been searched?
Are there folders whose permissions must be checked?
Are there executables whose permissions must be checked?
Does the agent know the current username?
Does the agent know the Windows path?
Are there base64-credentials to decode?

4.3 State of the Agent

We update the state of the agent by keeping the information that is relevant for the task of privilege escalation. The agent state includes variables that contain general information about the system, information about discovered services, dynamic-link libraries (DLLs), AutoRun registry, and scheduled tasks.

The general information is represented by the 27 variables listed in Table 1. Seven of these variables are trinary (true/unknown/false) and contain information useful for the task of privilege escalation. The remaining 20 variables are binary (true/false), and they also contain information about the previous actions of the agent. The previous actions are included in the state to make the agent state as close to Markov as possible, which makes the training easier.

At the beginning of each training episode, the agent has no knowledge of the services running on the host. The agent has to collect a list of services by taking the action *A31 Get a list of services*. Once a service is detected, it is described by its *name*, *full path*, the owning *user* and the 11 trinary attributes listed in Table 2. Each of these attributes can have three possible values: true (+1), unknown (0), and false (-1). Then, the agent needs to perform actions such as *A25 Check service permissions with accesschk64* to fill the values of the unknown attributes.

Since local privilege escalation can be performed by DLL hijacking, we also include the information about the DLLs used by the services in the state. Each DLL is described using a set of attributes listed in Table 2. This information is added to the state after taking action *A29 Analyze service executables for DLLs*.

Privileges can be elevated in Windows by using vulnerable executables in the AutoRun registry and misconfigured scheduled tasks. Therefore, we add information about the AutoRun files and the scheduled tasks to the agent state. Each AutoRun file and each scheduled task is described using the trinary attributes defined in Table 2.

Table 2: Part of the agent state with information about services, DLLs, AutoRuns, and scheduled tasks (all are trinary variables)

Service	Is the service running?
	Is the service run with elevated privileges?
	Is the service path unquoted?
	Is there a writable parent folder?
	Is there whitespace in the service path?
	Is the service binary in C:\Windows?
	Can the service executable be written?
	Can the service be re-configured?
	Can the service registry be modified?
	Does the service load a vulnerable DLL?
	Has the service been exploited?
DLL	Is the DLL missing?
	Is the DLL writable?
	Has the DLL been replaced with a malicious DLL?
AutoRun	Is the AutoRun file writable?
	Is the AutoRun file in C:\Windows?
Task	Is the task run with elevated privileges?
	Is the executable writable?
	Is the executable in C:\Windows?

Table 3: Examples of auxiliary information used to fill the command arguments

Service	Name, executable path, user
Executable	Path, linked DLLs
DLL	Name, executable calling, path
AutoRun	Executable path, trigger
Task	Name, executable path, trigger, user
Credentials	Username, password, plaintext
File system	Folders, executables, permissions

In addition to the variables defined in Table 1 and Table 2, the agent maintains a collection of auxiliary information in its memory. The information is needed to fill the arguments of the commands executed by the agent. Examples of the auxiliary information are given in Table 3. This information is gathered and updated based on the observations, that is, the outputs of the commands performed by the agent. The auxiliary information is not given as input to the neural network, and hence, it affects neither the policy $\pi(a_t|s_t)$ nor the value $V(s_t)$ directly.

4.4 Action Space

We designed the action space of the agent by including actions needed for gathering information about the victim Windows host and performing the privilege escalation techniques. The action space consists of 38 actions listed in Table 4. Although the action space is crafted for known privilege escalation vulnerabilities (which we consider unavoidable within the constraints of the current RL), there is no one-to-one relationship between the actions and vulnerabilities. Some actions are only relevant for specific vulnerabilities, whereas many others are more general and can be used in multiple scenarios (see Appendix B). Our general principle in constructing the action space has been to make the actions as atomic as possible while keeping the problem potentially solvable by the current RL.

The actions are defined on a high level, which means that their exact implementation can vary, for example, depending on the platform. For instance, action *A29 Analyze service executables for DLLs* can be implemented by static analysis of the Portable Executable files with an open-source analyzer to detect the loaded DLLs. The same action can be implemented using a custom analyzer or a script to download the executable and analyze it with Process Monitor. Our high-level action definition enables modifying the low-level implementations of the actions, such as changing the frameworks used, without affecting the trained agent. To create the necessary malicious executables, we use Kali Linux with Metasploit. The malicious DLLs needed for performing DLL hijacking are compiled manually. However, the low-level implementation of these commands can easily be changed if desired.

Each of the high-level actions is well-specified and can be performed using only a handful of standard Windows (cmd.exe and PowerShell) and Linux (zsh) commands. Many of the commands need arguments. For instance, to take the action *A9 Start an exploited service*, the name of the service must be specified. In this work, we automatically fill the arguments using the auxiliary information collected as discussed in Section 4.3. For example, one of the actions defined is *A28 Check directory permissions with icacls*. The agent maintains an internal list of directories that are of interest, and when the action to analyze the permissions of directories is selected, every directory on the list is scanned.

5 EXPERIMENTS

5.1 Simulator of a Windows 7 Virtual Machine

A key practical challenge for training a reinforcement learning agent to perform red teaming tasks is the slow simulation speed when performing actions on a real virtual machine. For example, running commands necessary for privilege escalation can take longer than a minute on a full-featured Windows 7 virtual machine, even if the agent acts optimally. At the beginning of training, when the agent selects actions very close to randomly, one episode of training on a real VM can last significantly longer. Moreover, each training episode requires a new virtual machine that has been configured with one of the available vulnerabilities. Provisioning and configuring a virtual machine in such a manner will further add to the time it would take to train the agent. Training a successful agent may require thousands of training episodes, which can take a prohibitively large amount of time when training on a real operating system. Developing an infrastructure to tackle the long simulation times on a real system is a significant challenge, and it is left outside the scope of this study.

To alleviate this issue, we implemented a simulated Python environment that emulates the behavior of a genuine Windows 7 operating system relevant to the privilege escalation task. The simulation consists of, among others, a file system with access controls, Windows registry, AutoRuns, scheduled tasks, users, executables,

Table 4: Actions

A1. Create a malicious executable in Kali Linux	A19. Change service registry to point to a malicious executable
A2. Create a malicious service executable in Kali Linux	A20. Change service registry to add the user to local administrators
A3. Compile a custom malicious DLL in Kali Linux	A21. Install a malicious MSI file
A4. Create a malicious MSI in Kali Linux	A22. Search for unattend* sysprep* unattended* files
A5. Download a malicious executable in Windows	A23. Decode base64 credentials
A6. Download a malicious service executable in Windows	A24. Test credentials
A7. Download a malicious DLL in Windows	A25. Check service permissions with accesschk64
A8. Download a malicious MSI in Windows	A26. Check the ACLs of the service registry with Get-ACL
A9. Start an exploited service	A27. Check executable permissions with icacls
A10. Stop an exploited service	A28. Check directory permissions with icacls
A11. Overwrite the executable of an autorun	A29. Analyze service executables for DLLs
A12. Overwrite the executable of a scheduled task	A30. Search for DLLs
A13. Overwrite a service binary	A31. Get a list of services
A14. Move a malicious executable so that it is executed by an unquoted service path	A32. Get a list of AutoRuns
	A33. Get a list of scheduled tasks
A15. Overwrite a DLL	A34. Check AlwaysInstallElevated bits
A16. Move a malicious DLL to a folder on Windows path to replace a missing DLL	A35. Check for passwords in Winlogon registry
	A36. Get a list of local users and administrators
A17. Re-configure service to use a malicious executable	A37. Get the current user
A18. Re-configure service to add the user to local administrators	A38. Get the Windows path

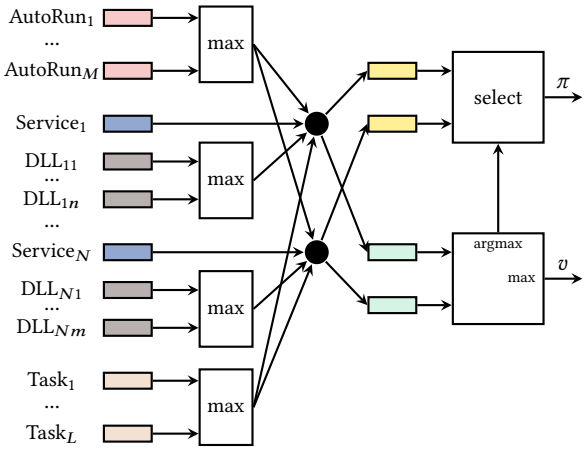

Figure 1: The architecture of the A2C agent. Colored boxes represent multilayer perceptrons (same colors denote shared parameters). The black circles represent the concatenation of input signals.

and services. Using this environment, the actions taken by the agent can be simulated in a highly efficient manner. Moreover, creating simulated machines with random vulnerabilities for training requires little programming and computing power and can be done very fast. However, to determine whether training the agent in a simulated environment instead of a real operating system is feasible, the trained agent will be evaluated by testing it on a vulnerable Windows 7 virtual machine.

5.2 The Architecture of the Agent

We use an A2C reinforcement learning agent (described in Section 3) in which the policy $\pi(a|s)$ and value $V(s)$ functions are modeled with neural networks. In practice, we use a single neural network with two heads: one produces the estimated value of the state $V(s)$ and the other the probabilities $\pi(a|s)$ of taking one of the 38 actions. The network gets as inputs the state variables described in Tables 1 and 2. The complete model has less than 27,000 parameters.

The main challenge in designing the neural network is that the number of AutoRuns, services, tasks, and DLLs can vary across training episodes. A Windows host might have anything from dozens to thousands of services, and the number of tasks and AutoRuns might also vary significantly depending on the host. We want our agent to be able to generalize to any number of those. Therefore, we process the information about each service, AutoRun, and scheduled task separately and aggregate the outputs of these computational blocks using the maximum operation. The architecture of the neural network is presented in Figure 1. Computational blocks with shared parameters are shown with the same colors.

In the proposed architecture, we concatenate the max-aggregated outputs of the blocks that process the information about AutoRuns, tasks, and DLLs with the outputs of the blocks that process the information about the individual services. Intuitively, this corresponds to augmenting the service data with the information about the most vulnerable AutoRun and task. Note that we also augment the service data with the information about the DLLs used by the corresponding service. Then, we pass the concatenated information through multilayer perceptrons that output value estimates and policies for all services. Finally, we regard the service with the highest value estimate as the most vulnerable one and select the policy corresponding to that service as the final policy.

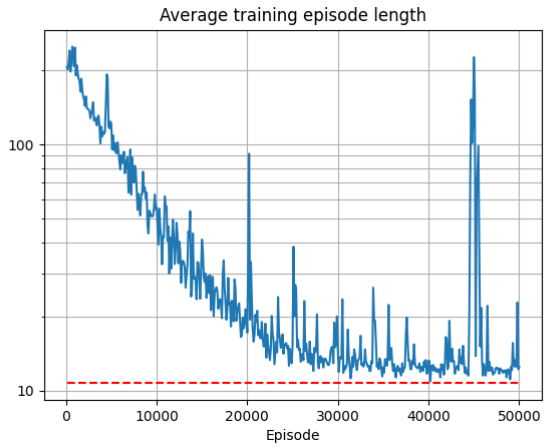

Figure 2: The average episode length during training on a logarithmic scale. The red line illustrates the episode length of an optimal policy that is approximately 10.7 actions per episode.

5.3 Training

Training consists of episodes in which the agent interacts with one instance of a sampled environment. At the beginning of each episode, the agent has no knowledge of the environment. The empty agent state is fed into the neural network that produces the value and the policy outputs. The action is sampled from the probabilities given by the policy output. Thus, we do not use explicit exploration strategies such as epsilon-greedy. The selected action is performed in the simulated environment, and the reward and the observations received as a result of the action are passed to the agent. The observations are parsed to update the agent state as described in Section 4.3. Then, a new action is selected based on the updated state. The iteration continues until the maximum number of steps for one episode is reached, or the agent has successfully performed privilege escalation.

The parameters of the neural network are updated at the end of each episode. The gradient (1) is computed automatically by back-propagation with PyTorch [34] and the parameters are updated using the Adam optimizer [27]. The agent is trained for as long as the average reward per episode continues to increase. The hyper-parameters used for training the agent are given in Appendix A.

Figure 2 presents the evolution of the episode length (averaged over 100 episodes) during one training run. The average episode length starts from around 200, and it gradually decreases reaching a level slightly above 11 after approximately 30,000 training episodes. We use 1,000 as the maximum number of steps per episode (see Appendix A), which implies that the agent manages to solve the problem and gets rewards from the very beginning of the optimization procedure when it takes close-to-random actions. We estimated that the average episode length is approximately 10.7 actions if the agent acts according to the optimal policy. Thus, the results indicate that the agent has learned to master the task of privilege escalation.

Training the agent for 50,000 episodes in the simulated environment (see Section 5.1) takes less than two hours without any significant code optimizations using a single NVIDIA GeForce GTX 1080 Ti GPU, which is a high-end consumer GPU from 2017. Note that performing the same training on a real Windows 7 virtual machine could take weeks.

5.4 Testing the Agent

Next, we test whether the agent trained in our simulated environment can transfer to a real Windows 7 operating system without any adaptation. We also compare the performance of our agent to two baselines: a ruleset crafted by an expert that can be interpreted as the optimal policy and a random policy.

We create a Windows 7 virtual machine using Hyper-V provided by the Windows 10 operating system. We assume that the offensive actor has gained low-level access with code execution rights by performing, for example, a successful penetration test or a phishing campaign. This is simulated by installing an SSH server on the victim host. We use the Paramiko SSH library in Python to connect to the virtual machine and execute commands with user-level credentials [17]. We use Hyper-V to create a Kali Linux virtual machine with Metasploit for generating malicious executables. However, instead of using Metasploit for creating malicious DLLs, the agent has to modify and compile a custom DLL code by taking action *A3 Compile a custom malicious DLL in Kali Linux*. Paramiko is also used to connect to the Kali machine.

Using SSH for simulating low-level code execution rights on the victim Windows 7 has some limitations. Some of the Windows command-line utilities such as wmic and accesschk64 are blocked by non-privileged users over SSH. To overcome this limitation of the test scenario, we open a second SSH session using elevated credentials and run the blocked commands in that session. In practice, a malicious actor would be able to execute these utilities while accessing the victim's environment via reverse shell or meterpreter session. Care was taken to prevent the test infrastructure from affecting the target environment. For example, due to the selection of an SSH tunnel as an off-the-shelf communication channel for testing purposes, the agent does not target any SSH-related vulnerabilities. Engineering effect was not prioritized for creating a production-ready attack agent, as it was considered beyond the scope of the research.

In order to take some of the actions listed in Table 4, the victim host has to have Windows Sysinternals with accesschk64. Moreover, we need an executable for scanning the DLLs loaded by the PE files. We used an open-source solution for that, but it failed to detect a DLL loaded by a handcrafted service executable. To work around this issue, we hard-coded the result of the scan in the agent. To properly address this issue, the high-level action of performing PE scanning could be mapped to a script that uploads the service executable on a Windows machine and uses ProcMon from Sysinternals to analyze the DLLs loaded by the service executable. Alternatively, a superior PE analyzer could be used.

First, we tested our agent on a virtual machine without external antivirus (AV) software or an intrusion detection system, but which had an up-to-date and active Windows Defender (which is essentially only an anti-spyware program in Windows 7). We kept the

Table 5: Actions to exploit a missing DLL file

A35. Check for passwords in Winlogon registry
A37. Get the current user
A31. Get a list of services
A28. Check directory permissions with icacls
A36. Get a list of local users and administrators
A26. Check the ACLs of service registries with Get-ACL
A25. Check service permissions with accesschk64
A34. Check AlwaysInstallElevated bits
A32. Get a list of AutoRuns
A28. Check directory permissions with icacls
A27. Check executable permissions with icacls
A22. Search for unattend* sysprep* unattended* files
A33. Get a list of scheduled tasks
A28. Check directory permissions with icacls
A27. Check executable permissions with icacls
A29. Analyze service executables for DLLs
A30. Search for DLLs
A28. Check directory permissions with icacls
A38. Get the Windows path
A28. Check directory permissions with icacls
A3. Compile a custom malicious DLL in Kali Linux
A7. Download a malicious DLL in Windows
A16. Move a malicious DLL to a folder on Windows path to replace a missing DLL
A9. Start an exploited service

Table 6: Actions to exploit a service with a missing binary

A35. Check for passwords in Winlogon registry
A37. Get the current user
A31. Get a list of services
A28. Check directory permissions with icacls
A2. Create a malicious service executable in Kali Linux
A6. Download a malicious service executable in Windows
A13. Overwrite a service binary
A9. Start an exploited service

number of services similar to the number of services during training by excluding all services in C:\Windows from the list of services gathered by the agent. We made the agent deterministically select the action with the highest probability. Our agent was successful in exploiting all the twelve vulnerabilities. Examples of the sequences of actions taken by the agent during evaluation can be found in Tables 5–8. The agent took very few unnecessary actions. The performance (measured in terms of the number of actions) could be improved by gathering more information before scanning for directory permissions. Now, the agent prefers scanning the directory permissions immediately after finding interesting directories. However, the amount of noise generated by the agent would have been similar as the agent would have performed more Windows commands per high-level action.

After that, we did not limit the number of services (by excluding services in C:\Windows) and let the agent perform privilege

Table 7: Actions to exploit elevated credentials in the WinLogon registry

A35. Check for passwords in Winlogon registry
A36. Get a list of local users and administrators
A24. Test credentials

Table 8: Actions to exploit AlwaysInstallElevated

A35. Check for passwords in Winlogon registry
A37. Get the current user
A31. Get a list of services
A28. Check directory permissions with icacls
A36. Get a list of local users and administrators
A26. Check the ACLs of service registries with Get-ACL
A25. Check service permissions with accesschk64
A34. Check AlwaysInstallElevated bits
A4. Create a malicious MSI file in Kali Linux
A8. Download a malicious MSI file in Windows
A21. Install a malicious MSI file

escalation. The increased number of services had no negative effect on the agent, and the agent was successful at the task. An example sequence of commands is given in Appendix C. However, because the agent performs each selected action on every applicable service, the agent generates some noise by scanning through the permissions of all services in C:\Windows. That could have caused an alert in an advanced detection and response system.

The number of actions used by the agent to escalate privileges during the testing phase is given in Table 9. We compare the following agents:

- the oracle agent, which assumes complete knowledge of the system, including the vulnerability;
- the optimal policy, which is approximated using a fixed ruleset crafted by an expert;
- the deterministic RL agent, which selects the action with the highest probability;
- the stochastic RL agent, which samples the action from the probabilities produced by the policy network;
- an agent taking random actions.

For all random trials, we used 1000 samples and computed the average number of actions used by the agent. Because of the computational cost of running thousands of episodes, all tests involving randomness were run in a simulated environment similar to the testing VM. The results suggest that the policy of the deterministic agent is close to optimal. The addition of stochasticity to action selection has a slightly negative effect on the performance but it increases the variability of the agent's actions making the agent potentially more difficult to detect.

We additionally tested the ability of the agent to generalize to multiple vulnerabilities which might be simultaneously present in the system. This was done in three ways. First, the agent was evaluated in an environment with six different types of vulnerable services present. Second, the agent was evaluated in an environment

Table 9: Number of actions used to escalate privileges

Vulnerability	Oracle (full knowledge)	Expert (≈ optimal policy)	Deterministic RL	Stochastic RL	Random
1	10	20	24	25.3	231.2
2	5	10	9	8.0	152.2
3	7	7	8	8.2	206.0
4	5	9	8	9.0	147.7
5	7	10	15	11.5	212.2
6	7	7	8	8.3	208.1
7	6	9	14	14.1	171.7
8	5	15	11	11.0	156.0
9	3	11	3	10.9	96.0
10	4	13	14	13.5	162.8
11	6	9	17	17.6	166.9
12	6	8	13	13.3	165.0
AVG	5.9	10.7	12.0	12.6	173.0

with all twelve vulnerabilities present. Finally, random combinations of any two vulnerabilities were tested. The agent had little trouble performing privilege escalation in any of these scenarios.

As a matter of interest, we finally evaluated the agent's performance against a host running an up-to-date version of a standard endpoint protection software, Microsoft Security Essentials, with real-time protection enabled. As expected, the AV software managed to recognize the default malicious executables created by msfvenom in Kali Linux. However, the AV software failed to recognize the custom DLL compiled by the agent, and hence, privilege escalation using DLL hijacking was possible. Moreover, the AV software failed to detect any methods that did not involve a downloaded malicious payload, such as re-configuring the vulnerable service to execute a command that added the user as a local administrator. Hence, privilege escalation was possible in many of the scenarios, even with up-to-date AV software present. It should be noted that these techniques fall beyond the scope of file-based threat detection used by standard antivirus software and would require more advanced protection strategies to counter, such as behavioral- or heuristics-based detection. The agent's performance against such detection engines was considered to be beyond the scope of the project and was not assessed.

6 DISCUSSION

Our work demonstrates that it is possible to train a deep reinforcement learning agent to perform local privilege escalation in Windows 7 using known vulnerabilities. Our method is the first reinforcement learning agent, to the best of our knowledge, that performs privilege escalation with an extensive state space and a high-level action space with easily customizable low-level implementations. Despite being trained in simulated environments, the test results demonstrate that our agent can solve the formalized privilege escalation problem in a close-to-optimal fashion on full-featured Windows machines with realistic vulnerabilities.

The efficacy of our implementation is limited if up-to-date antivirus software is running on the victim host because only a handcrafted DLL is used, whereas the malicious executables are created using Metasploit with default settings. However, if the mapping from the high-level actions to the low-level commands (see Table 4) was improved so that more sophisticated payloads were used or the action space was expanded with actions for defense evasion, a reinforcement learning agent could be capable of privilege escalation in hosts with up-to-date antivirus software but without an advanced detection and response system.

While simple attacks are likely to be detected by advanced breach detection solutions, not all companies employ those for various reasons. The constant stream of breaches seen in the news reflects that reality. Moreover, if adversaries develop RL-based tools for learning and automating adversarial actions, they might prefer to target networks that are less likely to be running breach detection software.

The current threat level presented by reinforcement learning agents is most likely limited to agents capable of exploiting existing *well-known* vulnerabilities. The same could be achieved by a scripted agent with a suitable hard-coded logic. However, the RL approach offers a number of benefits compared to a scripted agent:

- Scripting an attacking agent can be difficult when the number of potentially exploitable vulnerabilities grows and if the attacked system contains an IDS.
- The probabilistic approach of our RL agent will produce more varied attacks (and attempted attacks) than a scripted robot that follows hard-coded rules, which makes our agent more usable for training ML-based defenses and testing and evaluating intrusion detection systems.
- An RL agent may be quickly adapted to changes in the environment. For example, if certain sequences of actions cause an alarm raised by an intrusion detection system, the agent might learn to take a different route, which is not detectable by the IDS. This would produce invaluable information for strengthening the defense system.

In the long run, RL agents could have the potential to discover and exploit novel *unseen* vulnerabilities, which would have an enormous impact on the field. To implement this idea, agents would most likely need to interact with an authentic environment, which

would require a great deal of engineering effort and a huge amount of computational resources. Crafting the action space would nevertheless be most likely unavoidable within the constraints of the current RL methods. However, the development should go in the direction of making the actions more atomic and minimizing the amount of prior knowledge used in designing the action space. This could allow the agent to encompass more vulnerabilities and could be a way to get closer to the ultimate goal of discovering new vulnerabilities.

Another research direction is to increase the complexity of the learning task. In this first step, we wanted to understand how RL-powered attacks could work in a constrained, varied setup, and our key result is showing that the RL approach works for such a complex learning task. Defeating defensive measures or expanding to a wider range of target environments would be a research topic with a significantly larger scope. It would be interesting to see whether a reinforcement learning agent can perform more steps in the cyber security kill chain, such as defense evasion. It would also be interesting to train the agent in an environment with an intrusion detection system or a defensive RL agent and perform multi-agent reinforcement learning, which has been done in previous research on post-exploitation in simulated environments [12].

Ethical Considerations

The primary goal of this work is to contribute to building resistant defense systems that can detect and prevent various types of potential attacks. Rule-based defense systems can be effective, but as the number of attack scenarios grows, they become increasingly difficult to build and maintain. Data-driven defensive systems trained with machine learning offer a promising alternative but implementing this idea in practice is challenged by the problem of scarcity of available training data in this domain. In this work, we present a possible solution to this problem by training a reinforcement learning agent to perform malicious activities and therefore to generate invaluable training data for improving defense systems. The presented approach can also be useful to support the red teaming activities performed by cyber security experts by automating some steps in the kill chain. However, the system developed in this project can potentially be dangerous in the wrong hands. Hence, the code created in this project will not be open-sourced or released to the public.

ACKNOWLEDGMENTS

We thank the Security and Software Engineering Research Center S^2ERC for funding our work. In addition, we thank David Karpuk, Andrew Patel, Paolo Palumbo, Alexey Kirichenko, and Matti Aksela from F-Secure for their help in running this project and their domain expertise, Tuomas Aura for giving us highly valuable feedback, and the Academy of Finland for the support within the Flagship Programme Finnish Center for Artificial Intelligence (FCAI).

REFERENCES

[1] Mohammad Alauthman, Nauman Aslam, Mouhammd Al-Kasassbeh, Suleman Khan, Ahmad Al-Qerem, and Kim-Kwang Raymond Choo. 2020. An efficient reinforcement learning-based Botnet detection approach. *Journal of Network and Computer Applications* 150 (2020), 102479.

[2] Hyrum S Anderson, Anant Kharkar, Bobby Filar, David Evans, and Phil Roth. 2018. Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning. *arXiv preprint arXiv:1801.08917* (2018).

[3] Andy Applebaum, Doug Miller, Blake Strom, Chris Korban, and Ross Wolf. 2016. Intelligent, Automated Red Team Emulation. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*. 363–373.

[4] Giovanni Apruzzese, Michele Colajanni, Luca Ferretti, Alessandro Guido, and Mirco Marchetti. 2018. On the Effectiveness of Machine and Deep Learning for Cyber Security. In *2018 10th International Conference on Cyber Conflict (CyCon)*. IEEE, 371–390.

[5] John A Bland, Mikel D Petty, Tymaine S Whitaker, Katia P Maxwell, and Walter Alan Cantrell. 2020. Machine Learning Cyberattack and Defense Strategies. *Computers & Security* 92 (2020), 101738.

[6] Francesco Caturano, Gaetano Perrone, and Simon Pietro Romano. 2021. Discovering reflected cross-site scripting vulnerabilities using a multiobjective reinforcement learning environment. *Computers & Security* 103 (2021), 102204.

[7] Ünal Çavuşoğlu. 2019. A new hybrid approach for intrusion detection using machine learning methods. *Applied Intelligence* 49, 7 (2019), 2735–2761.

[8] Moitrayee Chatterjee and Akbar-Siami Namin. 2019. Detecting Phishing Websites through Deep Reinforcement Learning. In *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 2. IEEE, 227–232.

[9] Tong Chen, Jiqiang Liu, Yingxiao Xiang, Wenjia Niu, Endong Tong, and Zhen Han. 2019. Adversarial attack and defense in reinforcement learning-from AI security view. *Cybersecurity* 2, 11 (2019), 1–22.

[10] Ankur Chowdhary, Dijiang Huang, Jayasurya Sevalur Mahendran, Daniel Romo, Yuli Deng, and Abdulhakim Sabur. 2020. Autonomous Security Analysis and Penetration Testing. In *2020 16th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 508–515.

[11] Zhihua Cui, Fei Xue, Xingjuan Cai, Yang Cao, Gai-ge Wang, and Jinjun Chen. 2018. Detection of Malicious Code Variants Based on Deep Learning. *IEEE Transactions on Industrial Informatics* 14, 7 (2018), 3187–3196.

[12] Richard Elderman, Leon JJ Pater, Albert S Thie, Madalina M Drugan, and Marco Wiering. 2017. Adversarial Reinforcement Learning in a Cyber Security Simulation.. In *ICAART (2)*. 559–566.

[13] László Erdődi and Fabio Massimo Zennaro. 2021. The Agent Web Model: modeling web hacking for reinforcement learning. *International Journal of Information Security* (2021), 1–17.

[14] Zhiyang Fang, Junfeng Wang, Jiaxuan Geng, and Xuan Kan. 2019. Feature Selection for Malware Detection Based on Reinforcement Learning. *IEEE Access* 7 (2019), 176177–176187.

[15] Zhiyang Fang, Junfeng Wang, Boya Li, Siqi Wu, Yingjie Zhou, and Haiying Huang. 2019. Evading Anti-Malware Engines With Deep Reinforcement Learning. *IEEE Access* 7 (2019), 48867–48879.

[16] Aidin Ferdowsi, Ursula Challita, Walid Saad, and Narayan B Mandayam. 2018. Robust Deep Reinforcement Learning for Security and Safety in Autonomous Vehicle Systems. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 307–312.

[17] Jeff Forcier. 2021. *Paramiko: A Python Implementation of SSHv2.* http://www.paramiko.org/index.html

[18] Mohamed C Ghanem and Thomas M Chen. 2018. Reinforcement Learning for Intelligent Penetration Testing. In *2018 Second World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4)*. IEEE, 185–192.

[19] Mohamed C Ghanem and Thomas M Chen. 2020. Reinforcement Learning for Efficient Network Penetration Testing. *Information* 11, 1 (2020), 6.

[20] Andy Greenberg. 2018. The Untold Story of NotPetya, the Most Devastating Cyberattack in History. *Wired, August* 22 (2018).

[21] Guoan Han, Liang Xiao, and H Vincent Poor. 2017. Two-Dimensional Anti-Jamming Communication Based on Deep Reinforcement Learning. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2087–2091.

[22] Yi Han, Benjamin IP Rubinstein, Tamas Abraham, Tansu Alpcan, Olivier De Vel, Sarah Erfani, David Hubczenko, Christopher Leckie, and Paul Montague. 2018. Reinforcement Learning for Autonomous Defence in Software-Defined Networking. In *International Conference on Decision and Game Theory for Security*. Springer, 145–165.

[23] Xiaofan He, Huaiyu Dai, and Peng Ning. 2016. Faster Learning and Adaptation in Security Games by Exploiting Information Asymmetry. *IEEE Transactions on Signal Processing* 64, 13 (2016), 3429–3443.

[24] Danny Hendler, Shay Kels, and Amir Rubin. 2018. Detecting Malicious PowerShell Commands using Deep Neural Networks. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*. 187–197.

[25] Peter J Huber. 1992. Robust Estimation of a Location Parameter. In *Breakthroughs in Statistics*. Springer, 492–518.

[26] TaeGuen Kim, BooJoong Kang, Mina Rho, Sakir Sezer, and Eul Gyu Im. 2018. A Multimodal Deep Learning Method for Android Malware Detection using Various Features. *IEEE Transactions on Information Forensics and Security* 14, 3 (2018), 773–788.

[27] Diederik P Kingma and Jimmy Ba. 2014. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980* (2014).

[28] Ryusei Maeda and Mamoru Mimura. 2021. Automating post-exploitation with deep reinforcement learning. *Computers & Security* 100 (2021), 102108.

[29] Dean Richard McKinnel, Tooska Dargahi, Ali Dehghantanha, and Kim-Kwang Raymond Choo. 2019. A systematic literature review and meta-analysis on artificial intelligence in penetration testing and vulnerability assessment. *Computers & Electrical Engineering* 75 (2019), 175–188.

[30] Nikola Milosevic, Ali Dehghantanha, and Kim-Kwang Raymond Choo. 2017. Machine learning aided Android malware classification. *Computers & Electrical Engineering* 61 (2017), 266–274.

[31] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *International conference on machine learning*. PMLR, 1928–1937.

[32] Thanh Thi Nguyen and Vijay Janapa Reddi. 2019. Deep Reinforcement Learning for Cyber Security. *arXiv preprint arXiv:1906.05799* (2019).

[33] Zhen Ni and Shuva Paul. 2019. A Multistage Game in Smart Grid Security: A Reinforcement Learning Solution. *IEEE Transactions on Neural Networks and Learning Systems* 30, 9 (2019), 2684–2695.

[34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.). Curran Associates, Inc., 8024–8035. http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[35] Rapid7. 2021. Metasploit Framework. (2021). https://docs.rapid7.com/metasploit/msf-overview/

[36] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement Learning: An Introduction*. MIT press.

[37] Isao Takaesu. 2018. *Deep Exploit*. https://github.com/13o-bbr-bbq/machine_learning_security/tree/master/DeepExploit

[38] Microsoft 365 Defender Research Team. 2021. *Gamifying machine learning for stronger security and AI models*. https://www.microsoft.com/security/blog/2021/04/08/gamifying-machine-learning-for-stronger-security-and-ai-models/

[39] Xiaoyue Wan, Geyi Sheng, Yanda Li, Liang Xiao, and Xiaojiang Du. 2017. Reinforcement Learning Based Mobile Offloading for Cloud-Based Malware Detection. In *GLOBECOM 2017-2017 IEEE Global Communications Conference*. IEEE, 1–6.

[40] Yufei Wang, Zheyuan Ryan Shi, Lantao Yu, Yi Wu, Rohit Singh, Lucas Joppa, and Fei Fang. 2019. Deep Reinforcement Learning for Green Security Games with Real-Time Information. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 1401–1408.

[41] Liang Xiao, Yan Li, Guolong Liu, Qiangda Li, and Weihua Zhuang. 2015. Spoofing Detection with Reinforcement Learning in Wireless Networks. In *2015 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 1–5.

[42] Liang Xiao, Xiaoyue Wan, Canhuang Dai, Xiaojiang Du, Xiang Chen, and Mohsen Guizani. 2018. Security in Mobile Edge Caching with Reinforcement Learning. *IEEE Wireless Communications* 25, 3 (2018), 116–122.

[43] Fabio Massimo Zennaro and Laszlo Erdodi. 2021. Modeling Penetration Testing with Reinforcement Learning Using Capture-the-Flag Challenges: Trade-offs between Model-free Learning and A Priori Knowledge. *arXiv preprint arXiv:2005.12632* (2021).

A SUPPLEMENTARY MATERIAL FOR RL

The hyperparameters used for training are listed below:

Parameter	Explanation	Value
γ	Discount rate	0.995
α	Learning rate for Adam	0.001
β_1	First moment decay rate for Adam	0.9
β_2	Second moment decay rate for Adam	0.999
N_{max}	Maximum number of steps	1000
N_{serv}	Number of services	[1, 20]
N_{auto}	Number of autoruns	[1, 10]
N_{task}	Number of tasks	[1, 10]
M_{dlls}	Number of DLLs loaded per service	[1, 4]

B ACTIONS REQUIRED TO EXPLOIT THE VULNERABILITIES

The sequences of actions that can be used to exploit the twelve vulnerabilities are presented in this section. The actions used in multiple scenarios are marked with the blue color.

(1.1) Missing DLL:

A37. Get the current user
A31. Get a list of services
A29. Analyze service executables for DLLs
A30. Search for DLLs
A38. Get the Windows path
A28. Check directory permissions with icacls
A3. Compile a custom malicious DLL in Kali Linux
A7. Download a malicious DLL in Windows
A16. Move a malicious DLL to a folder on Windows path to replace a missing DLL
A9. Start an exploited service

(1.2) Writable DLL:

A37. Get the current user
A31. Get a list of services
A29. Analyze service executables for DLLs
A30. Search for DLLs
A27. Check executable permissions with icacls
A3. Compile a custom malicious DLL in Kali Linux
A7. Download a malicious DLL in Windows
A15. Overwrite a DLL
A9. Start an exploited service

(2) Re-configurable Service:

A37. Get the current user
A31. Get a list of services
A25. Check service permissions with accesschk64
A18. Re-configure service to add the user to local administrators
A9. Start an exploited service

(3) Unquoted Service Path:

A37. Get the current user
A31. Get a list of services
A28. Check directory permissions with icacls
A2. Create a malicious service executable in Kali Linux
A6. Download a malicious service executable in Windows
A14. Move a malicious executable so that it is executed by an unquoted service path
A9. Start an exploited service

(4) Modifiable ImagePath:

A37. Get the current user
A31. Get a list of services
A26. Check the ACLs of the service registry with Get-ACL
A20. Change service registry to add the user to local administrators
A9. Start an exploited service

(5) Writable Service Executable:

A37. Get the current user
A31. Get a list of services
A27. Check executable permissions with icacls
A2. Create a malicious service executable in Kali Linux
A6. Download a malicious service executable in Windows
A13. Overwrite a service binary
A9. Start an exploited service

(6) Missing Service Executable:

A37. Get the current user
A31. Get a list of services
A28. Check directory permissions with icacls
A2. Create a malicious service executable in Kali Linux
A6. Download a malicious service executable in Windows
A13. Overwrite a service binary
A9. Start an exploited service

(7) Writable AutoRun Executable:

A37. Get the current user
A32. Get a list of AutoRuns
A27. Check executable permissions with icacls
A1. Create a malicious executable in Kali Linux
A5. Download a malicious executable in Windows
A11. Overwrite the executable of an AutoRun

(8) AlwaysInstallElevated:

A37. Get the current user
A34. Check AlwaysInstallElevated bits
A4. Create a malicious MSI in Kali Linux
A8. Download a malicious MSI in Windows
A21. Install a malicious MSI file

(9) WinLogon Registry:

A35. Check for passwords in Winlogon registry
A36. Get a list of local users and administrators
A24. Test credentials

(10) Unattend File:

A22. Search for unattend* sysprep* unattended* files
A23. Decode base64 credentials
A36. Get a list of local users and administrators
A24. Test credentials

(11) Writable Task Binary:

A37. Get the current user
A33. Get a list of scheduled tasks
A27. Check executable permissions with icacls
A1. Create a malicious executable in Kali Linux
A5. Download a malicious executable in Windows
A12. Overwrite the executable of a scheduled task

(12) Writable Startup Folder:

A37. Get the current user
A32. Get a list of AutoRuns
A28. Check directory permissions with icacls
A1. Create a malicious executable in Kali Linux
A5. Download a malicious executable in Windows
A11. Overwrite the executable of an AutoRun

C COMMAND-LINE EXAMPLE

We exemplify our mapping from actions to commands by showing the commands taken by the agent to escalate privileges by exploiting a service with weak folder permissions and a missing binary (see Table 6). Note that all the commands disclosed below have been derived from public sources (given in the footnotes) and can be recreated by security practitioners. Furthermore, none of the commands are proprietary to F-Secure.

A35. Check for passwords in Winlogon registry:[1]
```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\
    CurrentVersion\Winlogon" /v DefaultUsername
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\
    CurrentVersion\Winlogon" /v DefaultPassword
```

A37. Get the current user:[2]
```
whoami
```

A31. Get a list of services:[3]
```
wmic service get name,pathname,startname,startmode,started
    /format:csv
```

A28. Check directory permissions with icacls:[2]
```
icacls.exe "c:\windows\system32"
icacls.exe "c:\windows"
. . . (15 rows skipped)
icacls.exe "c:\program files (x86)\microsoft\edge"
icacls.exe "c:\program files\missing file service"
icacls.exe "c:\program files"
. . . (4 rows skipped)
icacls.exe "c:\windows\system32\wbem"
icacls.exe "c:\program files\windows media player"
```

A2. Create a malicious service executable in Kali Linux:[4]
```
sudo -S msfvenom -p windows/exec CMD='net localgroup
    administrators user /add' -f exe-service -o java_updater_svc
```

A6. Download a malicious service executable in Windows:[5]
```
powershell.exe -command "Invoke-WebRequest -Uri
    '82.130.20.144/java_updater_service'
    -OutFile 'C:\Users\user\Downloads\java_updater_svc"
move /y "C:\Users\user\Downloads\java_updater_svc"
    "C:\Users\user\Downloads\java_updater_svc.exe"
```

A13. Overwrite a service binary:[4]
```
copy /y "C:\Users\user\Downloads\java_updater_svc.exe"
    "c:\program files\missing file service\missingservice.exe"
```

A9. Start an exploited service:[4]
```
sc start missingsvc
```

[1] https://github.com/sagishahar/lpeworkshop/blob/master/Lab%20Exercises%20Walkthrough%20-%20Windows.pdf
[2] https://sushant747.gitbooks.io/total-oscp-guide/content/privilege_escalation_windows.html
[3] https://book.hacktricks.xyz/windows/windows-local-privilege-escalation
[4] https://infosecwriteups.com/privilege-escalation-in-windows-380bee3a2842
[5] https://adamtheautomator.com/powershell-download-file/